```
    }
    while(!head.compare_exchange_strong(old_counter,new_counter,
        std::memory_order_acquire,std::memory_order_relaxed));

    old_counter.external_count=new_counter.external_count;
}
```

If the `compare_exchange_strong()` call succeeds, you know that the value read had the `ptr` field set to what's now stored in `old_counter`. Because the store in `push()` was a release operation, and this `compare_exchange_strong()` is an acquire operation, the store synchronizes with the load and you have a happens-before relationship. Consequently, the store to the `ptr` field in the `push()` happens before the `ptr->next` access in `pop()`, and you're safe.

Note that the memory ordering on the initial `head.load()` didn't matter to this analysis, so you can safely use `std::memory_order_relaxed` for that.

Next up, the `compare_exchange_strong()` to set `head` to `old_head.ptr->next`. Do you need anything from this operation to guarantee the data integrity of this thread? If the exchange succeeds, you access `ptr->data`, so you need to ensure that the store to `ptr->data` in the `push()` thread happens before the load. However, you already have that guarantee: the acquire operation in `increase_head_count()` ensures that there's a synchronizes-with relationship between the store in the `push()` thread and that compare/exchange. Because the store to `data` in the `push()` thread is sequenced before the store to `head` and the call to `increase_head_count()` is sequenced before the load of `ptr->data`, there's a happens-before relationship, and all is well even if this compare/exchange in `pop()` uses `std::memory_order_relaxed`. The only other place where `ptr->data` is changed is the very call to `swap()` that you're looking at, and no other thread can be operating on the same node; that's the whole point of the compare/exchange.

If the `compare_exchange_strong()` fails, the new value of `old_head` isn't touched until next time around the loop, and you already